



Data Structures Review

Algorithms and Big O Notation

- **Algorithms:** A set of unambiguous instructions for solving a problem or carrying out a task. Algorithms must finish in a finite number of steps and each step must be precise and unambiguous.
- **Big O Notation:** Describes the worst-case performance, or growth rate, of an algorithm as the input size increases. It focuses on how resource usage grows with input size, ignoring constant factors and lower-order terms.

Common Time Complexities

- **$O(1)$ - Constant Time:** Algorithm takes the same amount of time regardless of input size.

▼ Example Code

```
def check_even_or_odd(number):  
    if number % 2 == 0:  
        return 'Even'  
    else:  
        return 'Odd'
```

- **$O(\log n)$ - Logarithmic Time:** Time increases slowly as input grows. Common in algorithms that repeatedly reduce problem size by a fraction (like Binary Search).
- **$O(n)$ - Linear Time:** Running time increases proportionally to input size.

▼ Example Code

```
for grade in grades:  
    print(grade)
```

- **$O(n \log n)$ - Log-Linear Time:** Common time complexity of efficient sorting algorithms like Merge Sort and Quick Sort.
- **$O(n^2)$ - Quadratic Time:** Running time increases quadratically. Often seen in nested loops.

▼ Example Code

```
for i in range(n):  
    for j in range(n):  
        print("Hello, World!")
```

Space Complexity

- **$O(1)$ - Constant Space:** Algorithm uses same amount of memory regardless of input size.
- **$O(n)$ - Linear Space:** Memory usage grows proportionally with input size.
- **$O(n^2)$ - Quadratic Space:** Memory usage grows quadratically with input size.

Problem-Solving Techniques

- **Understanding the Problem:** Read the problem statement multiple times. Identify the input, expected output, and how to transform input to output.
- **Pseudocode:** High-level description of algorithm logic that is language-independent. Uses common written language mixed with programming constructs like `IF`, `ELSE`, `FOR`, `WHILE`.

▼ Example Code



Python Certification

Data Structures Review

```
FOR EACH character IN original_string:
    ADD character TO THE BEGINNING OF reversed_string
DISPLAY reversed_string
```

- **Edge Cases:** Specific, valid inputs that occur at the boundaries of what an algorithm should handle. Always consider and test edge cases.

Arrays

- **Static Arrays:** Have a fixed size determined at initialization. Elements stored in adjacent memory locations. Size cannot be changed during program execution.
- **Dynamic Arrays:** Can grow or shrink automatically during program execution. Handle resizing through automatic copying to larger arrays when needed.

Python Lists (Dynamic Arrays)

▼ Example Code

```
numbers = [3, 4, 5, 6]

# Access elements
numbers[0] # 3

# Update elements
numbers[2] = 16

# Add elements
numbers.append(7)
numbers.insert(3, 15) # Insert at specific index

# Remove elements
numbers.pop(2) # Remove at specific index
numbers.pop() # Remove last element
```

Time Complexities for Dynamic Arrays

- **Access:** $O(1)$
- **Insert at end:** $O(1)$ average, $O(n)$ when resizing needed
- **Insert in middle:** $O(n)$
- **Delete:** $O(n)$ for middle, $O(1)$ for end

Stacks

- **Stacks:** Last-In, First-Out (LIFO) data structure. Elements added and removed from the top only.
- **Push Operation:** Adding an element to the top of the stack. Time complexity: $O(1)$.
- **Pop Operation:** Removing an element from the top of the stack. Time complexity: $O(1)$.

▼ Example Code

```
# Using Python list as stack
stack = []

# Push operations
stack.append(1)
stack.append(2)
stack.append(3)
```



Queues

- **Queues:** First-In, First-Out (FIFO) data structure. Elements added to the back and removed from the front.
- **Enqueue Operation:** Adding an element to the back of the queue. Time complexity: $O(1)$.
- **Dequeue Operation:** Removing an element from the front of the queue. Time complexity: $O(1)$.

▼ Example Code

```
from collections import deque

# Using deque for efficient queue operations
queue = deque()

# Enqueue operations
queue.append(1)
queue.append(2)
queue.append(3)

# Dequeue operations
first_element = queue.popleft() # Returns 1
```

Linked Lists

- **Linked Lists:** Linear data structure where each node contains data and a reference to the next node. Nodes are connected like a chain.

Singly Linked Lists

- **Structure:** Each node has data and one reference to the next node.
- **Traversal:** Can only move forward from head to tail.
- **Head Node:** First node in the list, usually the only directly accessible node.
- **Tail Node:** Last node in the list, points to `None`.

Operations and Time Complexities

- **Insert at beginning:** $O(1)$
- **Insert at end:** $O(n)$ - must traverse to end
- **Insert in middle:** $O(n)$ - must traverse to position
- **Delete from beginning:** $O(1)$
- **Delete from end:** $O(n)$ - must traverse to find previous node
- **Delete from middle:** $O(n)$ - must traverse to find node

Doubly Linked Lists

- **Structure:** Each node has data and two references: next node and previous node.
- **Traversal:** Can move in both directions.
- **Memory:** Requires more memory than singly linked lists due to extra reference.

Hash Maps and Sets

Maps and Hash Maps

- **Map (Abstract Data Type):** Manages collections of key-value pairs. Every key must be unique, but values can be repeated.
- **Hash Map:** Concrete implementation of map ADT using hashing technique. Uses hash function to generate hash values for keys, which determine storage location in underlying array.

Python Dictionaries (Hash Maps)



```
my_dictionary = {
    "A": 1,
    "B": 2,
    "C": 3
}

# Alternative creation
my_dictionary = dict(A=1, B=2, C=3)

# Access and modify
value = my_dictionary["A"] # 1
my_dictionary["A"] = 4     # Update value
del my_dictionary["A"]     # Remove key-value pair

# Check membership
"C" in my_dictionary

# Get keys, values, items
my_dictionary.keys()
my_dictionary.values()
my_dictionary.items()
```

Time Complexities for Hash Maps

- Average case: $O(1)$ for insert, get, delete
- Worst case: $O(n)$ when many hash collisions occur

Sets

- **Sets:** Unordered collections of unique elements. No duplicates allowed, no specific order maintained.
- **Immutable Elements Only:** Sets can only contain immutable data types (numbers, strings, tuples) because hash values must remain constant.

▼ Example Code

```
# Creating sets
numbers = {1, 2, 3, 4}
empty_set = set() # Must use set(), not {}

# Add and remove elements
numbers.add(5)
numbers.remove(4) # Raises KeyError if not found
numbers.discard(4) # No error if not found

# Set operations
set_a = {1, 2, 3, 4}
set_b = {2, 3, 4, 5, 6}

# Union, intersection, difference, symmetric difference
set_a.union(set_b) # or set_a | set_b
set_a.intersection(set_b) # or set_a & set_b
set_a.difference(set_b) # or set_a - set_b
set_a.symmetric_difference(set_b) # or set_a ^ set_b
```



Python Certification

Data Structures Review

```
set_a.issuperset(set_b)
set_a.isdisjoint(set_b)
```

```
# Membership testing
```

```
5 in numbers
```

Time Complexities for Sets

- **Average case:** $O(1)$ for add, remove, membership testing
- **Worst case:** $O(n)$ due to hash collisions

Hash Collisions

- **Hash Collision:** Occurs when two different keys produce the same hash value.
- **Collision Resolution Strategies:**
 - **Chaining:** Each array index points to a linked list storing all elements with same hash value
 - **Open Addressing:** Search for next available index using predefined sequence

When to Use Each Data Structure

- **Lists:** When you need ordered, indexed access and don't know size in advance
- **Stacks:** For LIFO operations (undo functionality, expression evaluation, backtracking)
- **Queues:** For FIFO operations (task scheduling, breadth-first search)
- **Linked Lists:** When frequent insertion/deletion at beginning, unknown size, no random access needed
- **Hash Maps:** For fast key-value lookups, counting occurrences, caching
- **Sets:** For uniqueness checking, mathematical set operations, removing duplicates

Assignment

Review the Data Structures topics and concepts.

Submit

Ask for Help